From Cup to Cupboard: A Multi-Database Demo for Ingredient-Level Demand Forecasting

1st Sebastian Lieb
Institute of Information Systems
Universität Hamburg
Hamburg, Germany
Sebastian.Lieb@studium.uni-hamburg.de

2nd Timur Varli
Department of Informatics
Universität Hamburg
Hamburg, Germany
timur.varli@studium.uni-hamburg.de

3rd Felix Wissel
Department of Informatics
Universität Hamburg
Hamburg, Germany
felix.wissel@studium.uni-hamburg.de

***Abstract*—Small retailers such as cafés must repeatedly decide how much of each ingredient to purchase, balancing the risk of running out of popular items against the cost of over-ordering perishable stock. We present an interactive demo that forecasts daily ingredient demand for a café by combining structured per-cup sales records from PostgreSQL with semi-structured recipe data from MongoDB in a multi-database pipeline. Drink-level sales are forecast independently using additive Triple Exponential Smoothing and mapped to ingredient quantities through the recipe database, aggregating over all drinks that share an ingredient. Through a graphical interface, visitors can manually tune the model's smoothing parameters or let a grid search select them automatically, then inspect the resulting forecasts for individual ingredients across both a held-out evaluation period and the days beyond the available data. Beyond producing plausible purchasing forecasts, the demo lets visitors directly compare ingredients that draw on many drinks against ones that depend on only one or two, exposing firsthand how this difference affects forecast stability, and more generally where a common forecasting method remains reliable and where its assumptions break down.**

***Index Terms*—time series forecasting, Holt-Winters, exponential smoothing, demand forecasting, PostgreSQL, MongoDB**

I. Introduction

Cafés and other small retailers face the challenge of how much of each ingredient should be ordered for the coming days or weeks. Ordering too little risks running out of popular products and disappointing customers while ordering too much leads to waste and higher cost. Accurate demand forecasting directly addresses this problem by estimating future ingredient consumption from historical sales patterns [1].

Beyond the retail context, forecasting time-series data arises in a wide range of applications, including short-to-medium-term predictions of business metrics like sales, revenue and demand planning [2]. The coffee sales scenario used in this demo serves as an example of the broader family of exponential smoothing methods [3]. The project demonstrates how structured sales records and semi-structured recipe data can be combined in a multi-database architecture to result in a forecast of daily ingredient requirements. The system introduces a graphical user interface through which users can select an ingredient and adjust key model parameters like the three smoothing parameters α, β, γ and immediately observe their effect on the forecast.

II. System/Solution/Architecture Overview

Figure 1 shows the system architecture, a multi-database design that reflects the heterogeneous nature of the data: structured, transactional sales records on one side, and semi-structured recipes on the other.

Per-cup sales transactions are stored in a normalized **PostgreSQL** schema. Recipes are nested and vary in structure, with different ingredients, units, and steps per drink, so they are stored as documents in a hosted **MongoDB Atlas** cluster instead.

The forecasting core, written in **Java 17**, loads daily per-drink cup counts from PostgreSQL, filling any missing days with zero to obtain a contiguous series per drink. Each series is forecast independently using additive Triple Exponential Smoothing (Section II-A), producing both a held-out evaluation forecast and a forecast beyond the available data. The forecasted cup counts are then multiplied by ingredient quantities from MongoDB and aggregated across drinks that share an ingredient, such as milk used in lattes, cortados, and hot chocolate. The results, along with the historical amounts, are written to an `ingredient_forecast` table in PostgreSQL, keyed by date and ingredient.

The GUI is built in **Python 3** using `PyQt5` and `matplotlib`, and reads from `ingredient_forecast` via `psycopg2`. It stays purely presentational, triggering (re-)runs of the Java jar with user-chosen parameters and rendering the persisted results. A Python grid search over $\alpha, \beta, \gamma \in \{0.1, \ldots, 0.9\}$ offers an alternative to manual tuning, selecting whichever combination minimizes RMSE against the held-out period.

A. Forecasting Approach

Ingredient demand is derived from per-drink forecasts using **additive Triple Exponential Smoothing (Holt-Winters)**, which decomposes each series into level, trend, and weekly ($L = 7$) seasonal components:

$$
\begin{aligned}
l_t &= \alpha(y_t - s_{t-L}) + (1-\alpha)(l_{t-1} + b_{t-1}),\\
b_t &= \beta(l_t - l_{t-1}) + (1-\beta)b_{t-1},\\
s_t &= \gamma(y_t - l_t) + (1-\gamma)s_{t-L},
\end{aligned}
$$

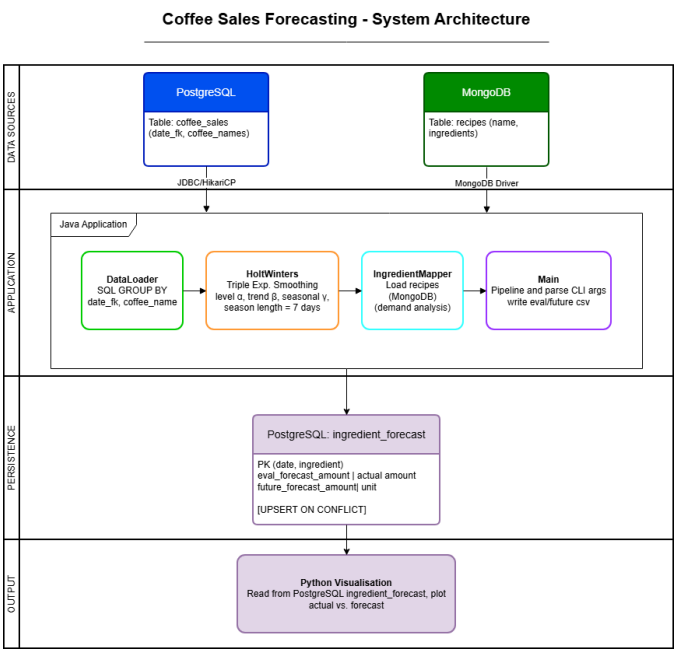

Fig. 1. Multi-database architecture: PostgreSQL stores sales data, MongoDB stores recipes, the Java core connects both, and the Python/PyQt5 GUI visualizes results.

With forecast $\hat{y}_{n+h} = l_n + hb_n + s_{n+h-L\lceil h/L \rceil}$. Negative forecasts are clipped to zero, and the level, trend, and seasonal terms are initialized from the mean and deviations of the first season.

This method was chosen over simpler alternatives because coffee sales show a clear weekly cycle alongside a slower yearly trend, both of which it captures with just three tunable parameters and closed-form updates. That makes it well suited to interactive tuning and to a grid search that needs to stay cheap. It is also a well-documented baseline for seasonal demand forecasting [1]–[3], which matches this ingredient-ordering use case.

III. Demonstration Proposal/Scenarios/Workflow

This section describes how users can experience our forecasting pipeline. Users can fine-tune the model manually or find suitable parameters automatically via a grid search, and see where the model behaves well as well as where its assumptions start to break down.

A. Setup

The demo runs on a laptop with an Apple Silicon M3 CPU and 16GB of RAM on macOS, however, the demo can also be run on less powerful hardware, given the moderate size of the dataset. The forecasting core is implemented in Java 17 and built with Maven into a self-contained jar. Historical sales are stored in PostgreSQL 16, while the ingredient recipes used to translate drink forecasts into ingredient quantities are kept in a hosted MongoDB Atlas cluster. Parameter search, visualization, and the graphical interface are implemented in Python 3, using `psycopg2` for database access, `pandas` for data handling, and `matplotlib` together with `PyQt5` for plotting and the GUI itself. A full grid search over the smoothing parameters α, β, γ at a step size of 0.1 evaluates $9^3 = 729$ parameter combinations per drink and completes within a few seconds on this hardware.

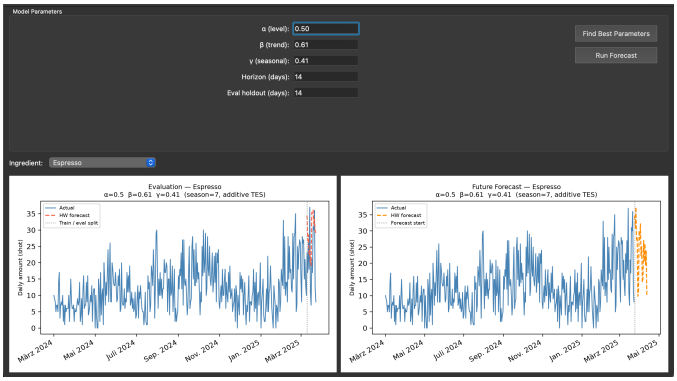

Fig. 2. The graphical interface showing model parameters, the ingredient selector, and the resulting evaluation and future forecast plots.

B. User Experience

Users can interact with the demo through a graphical interface (see Fig. 2). First, the user can select the parameters manually or run a grid search to find the best parameters for the forecasting model. Afterwards, the user can run the forecasting model and visualize the results for one ingredient. For the selected ingredient, two plots are created, one comparing actual sales against the forecast for the held-out evaluation period, and one showing the forecast for the upcoming days beyond the available data. After the forecast is generated and without running the forecasting model again, the user can select another ingredient in a drop-down menu and visualize the results for that ingredient. Visitors are also encouraged to compare ingredients that differ in how many drinks contribute to them. Ingredients drawn from several drinks tend to produce a stable forecast, since noise in any single drink's prediction is averaged out, whereas ingredients tied to only one or two drinks can inherit that drink's forecasting errors directly. This makes it possible to observe, within the same interface, both cases where the model performs convincingly and cases where its assumptions no longer hold.

C. Conclusion

The demo shows a complete path from raw, per-cup sales records to ingredient-level forecasts that could plausibly support purchasing decisions in a small coffee shop. Its particular value lies not in presenting a single polished result, but in letting visitors interactively expose the conditions under which a common forecasting method remains reliable and the conditions under which it does not, using the same pipeline and the same data throughout.

REFERENCES

[1] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. OTexts, Melbourne, Australia, 2018. [Online]. Available: https://otexts.com/fpp2/

[2] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," *International Journal of Forecasting*, vol. 20, no. 1, 2004.

[3] E. S. Gardner, "Exponential smoothing: The state of the art," *Journal of Forecasting*, vol. 4, no. 1, pp. 1–28, 1985.